

Architectural Reflection: Lab 3 — Composite Pattern

Course: Integrative Programming and Technologies (IPT)

Module: Structural Design Patterns in Web Application Development

Topic: Hierarchical Forum Trees & RAG Document Structures (Part-Whole Hierarchies)

1. Spotting the Smell: Tree Traversal via `instanceof` Branching

In `without_web.php`, the forum hierarchy relies on procedural tree rendering using type inspection:

```
function renderNaive(mixed $el, int $depth = 0): string {  
  
    if ($el instanceof PostNaive) { ... }  
  
    if ($el instanceof ThreadNaive) { ... }  
  
}
```

This pattern introduces severe structural deficiencies in web systems:

1. **Type Inspection Coupling:** The client traversal function must possess intimate knowledge of every concrete node class in the hierarchy.
 2. **Fragile Extensibility:** Introducing a new node type—such as a `Poll`, `PinnedNotice`, or `Bundle`—breaks `renderNaive()`, necessitating new conditional branches across every rendering and serialization routine.
 3. **Loss of Polymorphism:** Objects are treated as passive data structures rather than autonomous domain entities capable of rendering or embedding themselves.
-

2. Architectural Fix: The GoF Composite Pattern

The refactored implementation (`with_pattern.php`) adopts the **Composite Pattern**, establishing a shared polymorphic contract for both leaf nodes and container nodes:

- **Component Interface:** `ForumComponent` declares `display(int $depth = 0): void`.

- **Leaf Nodes (Post, Bundle):** Represent atomic domain units that implement `display()` directly without children.
- **Composite Nodes (Thread):** Contain a collection of `ForumComponent[]` children and implement `display()` by printing their own header and recursively invoking `$child->display($depth + 1)`.

The client interacts solely with the top-level `ForumComponent` contract. The distinction between an atomic post and an entire sub-thread of nested replies is completely encapsulated.

3. Academic Grounding (Gamma et al., 1994)

"Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly."

— Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (*Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, p. 163)

Buschmann et al. (1996) emphasize that the Composite pattern provides an elegant mechanism for **recursive decomposition**. In web architectures, part-whole relationships naturally model UI layouts, navigation structures, and content hierarchies where high-level containers must manipulate nested components without managing their internal types.

4. Extensibility Demonstration & Practical Web Applications

To prove extensibility, a new leaf element—**Bundle** (representing a pinned collection of course files)—was added with **1 class**. The composite `Thread` container displayed the bundle seamlessly with **zero modifications**. Furthermore, the pattern was ported to an LLM / Retrieval-Augmented Generation (RAG) domain:

- `TextComponent (getText(), embed())`
- `Chunk (Leaf) • Section (Composite) • Document (Composite)` The document recursively generates concatenated text and aggregated vector structures using the identical composite mechanism.

Real-World Web Applications in PHP Frameworks

- **View Component Trees (Laravel Blade / Symfony Twig):** Blade components (e.g., `<x-card>`, `<x-slot>`, `<x-button>`) form a recursive tree rendered via a single root compilation pass.

- **E-Commerce Catalog & Cart:** Categories contain subcategories and products; carts compute totals by uniformly iterating line items, product bundles, and discounts.
- **CMS Block Builders (Gutenberg / Elementor):** Page layouts composed of Columns → Rows → Blocks (Text, Image, Video) where each block implements a uniform `renderHtml()` interface.